

Curso de Ciência da Computação

Programação Orientada a

Objetos – POO

INTRODUÇÃO AO JAVA

Profa. Fernanda dos Santos Cunha

Programas exemplo retirados de **Java: como programar**
Deitel, P.; Deitel, H. - São Paulo: Pearson Education do Brasil, 2017.

MATERIAL REF. AULA HOJE



LIVROS BIBLIOTECA DIGITAL

SCHILDT, Herbert. Java para iniciantes. 6ª ed. 2015.
(Caps 1, 2, 3, 5 – linguagem)

LIVRO BIBLIOTECA FÍSICA

DEITEL, Paul J.; DEITEL, Harvey M. Java™: como programar. 10ª ed. 2016. (11ª ed. em inglês, 2017)
(Caps 1, 2, 4, 5, 7)

FERRAMENTA

IntelliJ IDEA - <https://www.jetbrains.com/idea/>

Programa Java

Elementos básicos:

Pacotes

```
import java.util
```

Classes

```
public class HelloJavaClass {
```

```
    public static void main (String args[]) {  
        System.out.println("Hello, Java");  
        Date d = new Date();  
        System.out.println("Date: " + d.toString());  
    }
```

Variáveis
(objetos)

Métodos

O método main é o ponto de entrada obrigatório para a execução de qualquer aplicação. Ele deve ser `public static void main(String[] args)` para permitir que a JVM inicie o programa, execute sem instanciar a classe, não retorne valor e aceite argumentos de linha de comando.

Pacotes Java

- Arquivos Java serão armazenados fisicamente em uma pasta
- Usando pacotes (*packages*) pode-se organizar de forma física um grupo de classes (lógica), mas elas devem ter forte relação entre si e com objetivo comum (ALTA COESÃO)
 - ✓ Uma mesma mudança deveria impactar em várias classes de um mesmo pacote
- Um pacote pode conter códigos-fonte, bibliotecas, arquivos de dados, outros pacotes

Packages Java

- Para indicar que as definições de um arquivo fonte Java fazem parte de um pacote, a 1ª linha de código deve ser a declaração de pacote:
`package nome_do_pacote;`
- Interessante usar URL invertida da organização, garantindo nomes únicos – **adotar:**
`package br.univali.poo;`
- Se esta declaração não estiver presente, as classes farão parte do pacote padrão que está mapeado para o diretório corrente

Programa Java

```
1 // Figura 2.1: Welcome1.java
2 // Programa de impressão de texto.
3
4 public class Welcome1
5 {
6     // método main inicia a execução do aplicativo Java
7     public static void main(String[] args)
8     {
9         System.out.println("Welcome to Java Programming!");
10    } // fim do método main
11 } // fim da classe Welcome1
```

Welcome to Java Programming!

```
1 // Figura 2.4: Welcome3.java
2 // Imprimindo múltiplas linhas de texto com uma única instrução.
3
4 public class Welcome3
5 {
6     // método main inicia a execução do aplicativo Java
7     public static void main(String[] args)
8     {
9         System.out.println("Welcome\nto\nJava\nProgramming!");
10    } // fim do método main
11 } // fim da classe Welcome3
```

Welcome
to
Java
Programming!

Programa Java

Entrada de Dados



- Um `Scanner` permite a leitura de dados a partir de várias origens (teclado, arquivo...)
- Deve-se criá-lo e especificar a origem dos dados:
`Scanner input = new Scanner(System.in) ;`
✓ `input` é o objeto `Scanner` (dar qualquer nome a ele)
- O objeto de entrada padrão `System.in` permite que aplicativos leiam bytes de informações digitadas pelo usuário
- O `Scanner` traduz esses bytes em tipos que podem ser utilizados em um programa

Programa Java

Entrada de Dados



- A partir do objeto `Scanner`, pode-se fazer estas leituras:

```
int x = input.nextInt();  
float y = input.nextFloat();  
double z = input.nextDouble();  
String s = input.nextLine();  
char c = input.next().charAt(0);
```

- Para ler um caractere pode-se usar também o método `read()` do pacote `System.in`:

```
char c = (char) System.in.read();
```


Programa Java

Saída de Dados



- Várias possibilidades de apresentação:

`System.out.print();`

- ✓ Gera uma saída de texto (String) e posiciona o cursor ao final da linha

`System.out.println();`

- ✓ Age semelhante ao `print`, mas pelo `ln` cria uma nova linha e posicionando o cursor abaixo do texto

`System.out.printf();`

- ✓ Gera uma saída formatada (`print formatted`), que pode consistir em texto fixo e especificadores de formato (`%b` boolean, `%d` int, `%f` float, `%2f` double, `%c` char, `%s` String)

Programa Java

Exemplo de E/S

```
1 // Figura 2.7: Addition.java
2 // Programa de adição que insere dois números, então exibe a soma deles.
3 import java.util.Scanner; // programa utiliza a classe Scanner
4
5 public class Addition
6 {
7     // método main inicia a execução do aplicativo Java
8     public static void main(String[] args)
9     {
10         // cria um Scanner para obter entrada a partir da janela de comando
11         Scanner input = new Scanner(System.in);
12
13         int number1; // primeiro número a somar
14         int number2; // segundo número a somar
15         int sum; // soma de number1 e number2
16
17         System.out.print("Enter first integer: "); // prompt
18         number1 = input.nextInt(); // lê primeiro o número fornecido pelo usuário
19
20         System.out.print("Enter second integer: "); // prompt
21         number2 = input.nextInt(); // lê o segundo número fornecido pelo usuário
22
23         sum = number1 + number2; // soma os números, depois armazena o total em sum
24
25         System.out.printf("Sum is %d\n", sum); // exibe a soma
26     } // fim do método main
27 } // fim da classe Addition
```

```
Enter first integer: 45
Enter second integer: 72
Sum is 117
```

Java

Tipos Numéricos



INTEIRO

Intervalo de representação

byte	8 bits	-128 .. 127
short	16 bits	-32.768 .. 32.767
int	32 bits	-2.147.483.648 .. 2.147.483.647
long	64 bits	-9.223.372.036.854.775.808 .. 9.223.372.036.854.775.807

LITERAIS NUMÉRICAS => por padrão são int

```
int x=0;
```

```
long y=0L; //para identificar long
```

Java

Tipos Numéricos



REAL/PONTO FLUTUANTE **Interv. de represent.**

float 32 bits, precisão simples, 7 dígitos
-1.40239846E-45 .. 3.40282347E+38

double 64 bits, dupla precisão, 15 dígitos
-4.94065645841246544E-324 .. 1.79769313486231570E+308

LITERAIS NUMÉRICAS => por padrão são double

double x=0.5;

float y=0.5**f**, z=0.3**F**; //p/identificar float

Java

Tipo Caractere



- Tipo `char` permite a representação de caracteres individuais
- Ocupa 16 bits permitindo até 32.768 caracteres diferentes (tabela ASCII)
- Valor padrão é o código zero: `'\0'` ou `'\u0000'`
- Pode representar um número inteiro de 16 bits sem sinal (0 a 65535)
- Caracteres de controle e outros de uso reservado devem ser usados precedidos por `< \ >`

Java

Tipo Caractere - Escape

<code>\b</code>	backspace
<code>\t</code>	tabulação horizontal
<code>\n</code>	newline
<code>\f</code>	form feed
<code>\r</code>	carriage return
<code>\"</code>	aspas
<code>\'</code>	aspas simples
<code>\\</code>	contrabarra
<code>\xxx</code>	o carácter com código de valor octal xxx, que pode assumir valores entre 000 e 377 na representação octal
<code>\uxxxx</code>	o carácter Unicode com código de valor hexadecimal xxxx, onde xxxx pode assumir valores entre 0000 e ffff na representação hexadecimal.

Java

Tipo Lógico



- Representado pelo tipo `boolean`
- Assume valores `false` (falso) ou `true` (verdadeiro)
- Valor padrão é `false`
- Ocupa 1 bit
- Usado em `if`, `switch`, `while` e `for`
- Não podem ser comparados com variáveis ou constantes numéricas

String Java

- **É uma classe**, não um tipo primitivo
- Não é preciso usar o construtor
`String name = "Ana Maria";`
`System.out.println(name);`
- Literais como `"Ana Maria"` são objetos (instâncias) da classe `String`
- Objetos `String` são imutáveis – não podem ter seu estado alterado após sua instanciação

String Java

Operadores para manipulação

```
String str= "Ola";  
if("Ola" == "Ola") { ... }           // comparacao  
if(str.equals("Ola")) { ... }  
if(str.equalsIgnoreCase("OLA")) { ... }  
  
str.length()           // pega o tamanho  
str.toUpperCase()      // converte para maiusculo  
str.charAt(1)          // pega caracter nesta posicao  
str.concat("DEF")      // concatena  
str.substring(1,2)     // pega substring no intervalo  
str.contains("ab")     // verifica se contém  
str.replace("a", "x")  // substituir elementos
```

Palavras reservadas Java

<i>abstract</i>	<i>continue</i>	<i>finally</i>	<i>interface</i>	<i>public</i>	<i>throw</i>
<i>boolean</i>	<i>default</i>	<i>float</i>	<i>long</i>	<i>return</i>	<i>throws</i>
<i>break</i>	<i>do</i>	<i>for</i>	<i>native</i>	<i>short</i>	<i>transient</i>
<i>byte</i>	<i>double</i>	<i>if</i>	<i>new</i>	<i>static</i>	<i>true</i>
<i>case</i>	<i>else</i>	<i>implements</i>	<i>null</i>	<i>super</i>	<i>try</i>
<i>catch</i>	<i>extends</i>	<i>import</i>	<i>package</i>	<i>switch</i>	<i>void</i>
<i>char</i>	<i>false</i>	<i>instanceof</i>	<i>private</i>	<i>synchronized</i>	<i>while</i>
<i>class</i>	<i>final</i>	<i>int</i>	<i>protected</i>	<i>this</i>	

Existem outras palavras reservadas que **NÃO SÃO USADAS** pela linguagem:

<i>const</i>	<i>future</i>	<i>generic</i>	<i>goto</i>	<i>inner</i>	<i>operator</i>
<i>outer</i>	<i>rest</i>	<i>var</i>	<i>volatile</i>		

Java

Declaração de variáveis



- Uma variável **não pode utilizar como nome uma palavra reservada** da linguagem

- Sintaxe:

`tipo identificador1; OU tipo var1, var2;`

- Exs.:

`int i;`

`float custo, precoTotal;`

`byte mascara;`

`double valorMedio;`

Java

Declaração de Classe

Visibilidade
da classe

Atributos
privados

Métodos
públicos

```
1 // Figura 3.1: Account.java
2 // Classe Account que contém uma variável de instância name
3 // e métodos para configurar e obter seu valor.
4
5 public class Account
6 {
7     private String name; // variável de instância
8
9     // método para definir o nome no objeto
10    public void setName(String name)
11    {
12        this.name = name; // armazena o nome
13    }
14
15    // método para recuperar o nome do objeto
16    public String getName()
17    {
18        return name; // retorna valor do nome para o chamador
19    }
20 } // fim da classe Account
```

Java

Declaração de objetos

Declaração
de objetos

```
1 // Figura 3.2: AccountTest.java
2 // Cria e manipula um objeto Account.
3 import java.util.Scanner;
4
5 public class AccountTest
6 {
7     public static void main(String[] args)
8     {
9         // cria um objeto Scanner para obter entrada a partir da janela de comando
10        Scanner input = new Scanner(System.in);
11
12        // cria um objeto Account e o atribui a myAccount
13        Account myAccount = new Account();
14
15        // exibe o valor inicial do nome (null)
16        System.out.printf("Initial name is: %s\n\n", myAccount.getName());
17
18        // solicita e lê o nome
19        System.out.println("Please enter the name:");
20        String theName = input.nextLine(); // lê uma linha de texto
21        myAccount.setName(theName); // insere theName em myAccount
22        System.out.println(); // gera saída de uma linha em branco
23
24        // exibe o nome armazenado no objeto myAccount
25        System.out.printf("Name in object myAccount is:\n%s\n",
26            myAccount.getName());
27    }
28 } // fim da classe AccountTest
```

Java

Operadores Aritméticos

Operador	Significado	Exemplo
+	Adição	$a + b$
-	Subtração	$a - b$
*	Multiplicação	$a * b$
/	Divisão	a / b
%	Resto da divisão inteira	$a \% b$
-	Sinal negativo (- unário)	$-a$
+	Sinal positivo (+ unário)	$+a$
++	Incremento unitário	$++a$ ou $a++$
--	Decremento unitário	$--a$ ou $a--$

Java

Operadores Relacionais

Operador	Significado	Exemplo
==	Igual	<code>a == b</code>
!=	Diferente	<code>a != b</code>
>	Maior que	<code>a > b</code>
>=	Maior ou igual a	<code>a >= b</code>
<	Menor que	<code>a < b</code>
<=	Menor ou igual a	<code>a <= b</code>

Operadores Lógicos

Operador	Significado	Exemplo
&&	E lógico (<i>and</i>)	<code>a && b</code>
	Ou Lógico (<i>or</i>)	<code>a b</code>
!	Negação (<i>not</i>)	<code>!a</code>

Java

Operadores Bitwise



<code>z = ~x</code>	<code>// Inverte os bits de x</code>
<code>z = x & y</code>	<code>// AND bit a bit entre x e y</code>
<code>z = x y</code>	<code>// OR bit a bit entre x e y</code>
<code>z = x ^ y</code>	<code>// XOR bit a bit entre x e y</code>
<code>z = x << y</code>	<code>// Desloca bits de x para esquerda, y vezes</code>
<code>z = x >> y</code>	<code>// Desloca bits de x para direita, y vezes</code>
<code>z = x >>> y</code>	<code>// Preenche zeros a esquerda de x, y vezes</code>

Java

Conversão



Conversão implícita

```
int x = 10;  
long y = x;           // OK pq int é menor que long  
x = y;              // erro: um long não "cabe" em int
```

Conversão explícita

```
long x = 10L;  
int y = (int)x; // agora sim!!! (cast)
```

Java

Estruturas de Controle de Fluxo

```
if (condicao) {  
    bloco_comandos  
}
```

```
switch (variavel) {  
    case valor1:  
        bloco_comandos  
        break;  
    case valor2:  
        bloco_comandos  
        break;  
    ...  
    case valorn:  
        bloco_comandos  
        break;  
    default:  
        bloco_comandos  
}
```

Java

Estruturas de Controle de Fluxo

```
while (condicao) {  
    bloco_comandos  
}
```

```
do {  
    bloco_comandos  
}while (condicao);
```

```
for (inicializacao; condicao; incremento) {  
    bloco_comandos  
}
```

Programa Java



UNIVALI

Exemplo de if

```
1 // Figura 4.4: Student.java
2 // Classe Student que armazena o nome e a média de um aluno.
3 public class Student
4 {
5     private String name;
6     private double average;
7
8     // construtor inicializa variáveis de instância
9     public Student(String name, double average)
10    {
11        this.name = name;
12
13        // valida que a média é > 0.0 e <= 100.0; caso contrário.
14        // armazena o valor padrão da média da variável de instância (0.0)
15        if (average > 0.0)
16        {
17            if (average <= 100.0)
18            {
19                this.average = average; // atribui à variável de instância
20            }
21        }
22
23        // define o nome de Student
24        public void setName(String name)
25        {
26            this.name = name;
27        }
28
29        // recupera o nome de Student
30        public String getName()
31        {
32            return name;
33        }
34    }
35 }
```

Construtor
com
parâmetros

Programa Java



Exemplo de if

```
31
32 // define a média de Student
33 public void setAverage(double studentAverage)
34 {
35     // valida que a média é > 0.0 e <= 100.0; caso contrário,
36     // armazena o valor atual da média da variável de instância
37     if (average > 0.0)
38         if (average <= 100.0)
39             this.average = average; // atribui à variável de instância
40 }
41
42 // recupera a média de Student
43 public double getAverage()
44 {
45     return average;
46 }
47
48 // determina e retorna a letra da nota de Student
49 public String getLetterGrade()
50 {
51     String letterGrade = ""; // inicializado como uma String vazia
52
53     if (average >= 90.0)
54         letterGrade = "A";
55     else if (average >= 80.0)
56         letterGrade = "B";
57     else if (average >= 70.0)
58         letterGrade = "C";
59     else if (average >= 60.0)
60         letterGrade = "D";
61     else
62         letterGrade = "F";
63
64     return letterGrade;
65 }
66 } // finaliza a classe Student
```

Programa Java

Exemplo de if

```
1 // Figura 4.5: StudentTest.java
2 // Cria e testa objetos Student.
3 public class StudentTest
4 {
5     public static void main(String[] args)
6     {
7         Student account1 = new Student("Jane Green", 93.5);
8         Student account2 = new Student("John Blue", 72.75);
9
10        System.out.printf("%s's letter grade is: %s\n",
11                           account1.getName(), account1.getLetterGrade());
12        System.out.printf("%s's letter grade is: %s\n",
13                           account2.getName(), account2.getLetterGrade());
14    }
15 } // fim da classe StudentTest
```

```
Jane Green's letter grade is: A
John Blue's letter grade is: C
```

Exemplo de while

```
1 // Figura 4.10: ClassAverage.java
2 // Resolvendo o problema da média da classe usando a repetição controlada por sentinela.
3 import java.util.Scanner; // programa utiliza a classe Scanner
4
5 public class ClassAverage
6 {
7     public static void main(String[] args)
8     {
9         // cria Scanner para obter entrada a partir da janela de comando
10        Scanner input = new Scanner(System.in);
11
12        // fase de inicialização
13        int total = 0; // inicializa a soma das notas
14        int gradeCounter = 0; // inicializa o nº de notas inseridas até agora
15
16        // fase de processamento
17        // solicita entrada e lê a nota do usuário
18        System.out.print("Enter grade or -1 to quit: ");
19        int grade = input.nextInt();
20
21        // faz um loop até ler o valor de sentinela inserido pelo usuário
22        while (grade != -1)
23        {
24            total = total + grade; // adiciona grade a total
25            gradeCounter = gradeCounter + 1; // incrementa counter
26
27            // solicita entrada e lê a próxima nota fornecida pelo usuário
28            System.out.print("Enter grade or -1 to quit: ");
29            grade = input.nextInt();
30        }
31
32        // fase de término
33        // se usuário inseriu pelo menos uma nota...
34        if (gradeCounter != 0)
35        {
36            // usa número com ponto decimal para calcular média das notas
37            double average = (double) total / gradeCounter;
38
39            // exibe o total e a média (com dois dígitos de precisão)
40            System.out.printf("\nTotal of the %d grades entered is %d\n",
41                               gradeCounter, total);
42            System.out.printf("Class average is %.2f\n", average);
43        }
44        else // nenhuma nota foi inserida, assim gera a saída da mensagem apropriada
45            System.out.println("No grades were entered");
46    }
47 } // fim da classe ClassAverage
```

```
Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1
```

```
Total of the 3 grades entered is 257
Class average is 85.67
```

Exemplo de switch

```
1 // Figura 5.11: AutoPolicy.java
2 // Classe que representa uma apólice de seguro de automóvel.
3 public class AutoPolicy
4 {
5     private int accountNumber; // número da conta da apólice
6     private String makeAndModel; // carro ao qual a apólice é aplicada
7     private String state; // abreviatura do estado com duas letras
8
9     // construtor
10    public AutoPolicy(int accountNumber, String makeAndModel, String state)
11    {
12        this.accountNumber = accountNumber;
13        this.makeAndModel = makeAndModel;
14        this.state = state;
15    }
16
17    // define o accountNumber
18    public void setAccountNumber(int accountNumber)
19    {
20        this.accountNumber = accountNumber;
21    }
22
23    // retorna o accountNumber
24    public int getAccountNumber()
25    {
26        return accountNumber;
27    }
28
29    // configura o makeAndModel
30    public void setMakeAndModel(String makeAndModel)
31    {
32        this.makeAndModel = makeAndModel;
33    }
34
```


Exemplo de switch

```
35 // retorna o makeAndModel
36 public String getMakeAndModel()
37 {
38     return makeAndModel;
39 }
40
41 // define o estado
42 public void setState(String state)
43 {
44     this.state = state;
45 }
46
47 // retorna o estado
48 public String getState()
49 {
50     return state;
51 }
52
53 // método predicado é retornado se o estado tem seguros "sem culpa"
54 public boolean isNoFaultState()
55 {
56     boolean noFaultState;
57
58     // determina se o estado tem seguros de automóvel "sem culpa"
59     switch (getState()) // obtém a abreviatura do estado do objeto AutoPolicy
60     {
61         case "MA": case "NJ": case "NY": case "PA":
62             noFaultState = true;
63             break;
64         default:
65             noFaultState = false;
66             break;
67     }
68
69     return noFaultState;
70 }
71 } // fim da classe AutoPolicy
```



UNIVALI

Exemplo de switch

```
1 // Figura 5.12: AutoPolicyTest.java
2 // Demonstrando Strings em um switch.
3 public class AutoPolicyTest
4 {
5     public static void main(String[] args)
6     {
7         // cria dois objetos AutoPolicy
8         AutoPolicy policy1 =
9             new AutoPolicy(11111111, "Toyota Camry", "NJ");
10        AutoPolicy policy2 =
11            new AutoPolicy(22222222, "Ford Fusion", "ME");
12
13        // exibe se cada apólice está em um estado "sem culpa"
14        policyInNoFaultState(policy1);
15        policyInNoFaultState(policy2);
16    }
17
18    // método que mostra se um AutoPolicy
19    // está em um estado com seguro de automóvel "sem culpa"
20    public static void policyInNoFaultState(AutoPolicy policy)
21    {
22        System.out.println("The auto policy:");
23        System.out.printf(
24            "Account #: %d; Car: %s; State %s %s a no-fault state\n\n",
25            policy.getAccountNumber(), policy.getMakeAndModel(),
26            policy.getState(),
27            (policy.isNoFaultState() ? "is": "is not"));
28    }
29 } // fim da classe AutoPolicyTest
```

The auto policy:
Account #: 11111111; Car: Toyota Camry;
State NJ is a no-fault state

The auto policy:
Account #: 22222222; Car: Ford Fusion;
State ME is not a no-fault state

```
1 // Figura 6.6: RandomIntegers.java
2 // Inteiros aleatórios deslocados e escalonados.
3 import java.security.SecureRandom; // o programa usa a classe SecureRandom
4
5 public class RandomIntegers
6 {
7     public static void main(String[] args)
8     {
9         // o objeto randomNumbers produzirá números aleatórios seguros
10        SecureRandom randomNumbers = new SecureRandom();
11
12        // faz o loop 20 vezes
13        for (int counter = 1; counter <= 20; counter++)
14        {
15            // seleciona o inteiro aleatório entre 1 e 6
16            int face = 1 + randomNumbers.nextInt(6);
17
18            System.out.printf("%d ", face); // exibe o valor gerado
19
20            // se o contador for divisível por 5, inicia uma nova linha de saída
21            if (counter % 5 == 0)
22                System.out.println();
23        }
24    }
25 } // fim da classe RandomIntegers
```

Exemplo
de for

```
1 5 3 6 2
5 2 6 5 2
4 4 4 2 6
3 1 6 2 2
```

Java

- A classe `Math` faz parte do pacote `java.lang`

Método	Descrição	Exemplo
<code>abs(x)</code>	valor absoluto de x	<code>abs(23.7)</code> é 23.7 <code>abs(0.0)</code> é 0.0 <code>abs(-23.7)</code> é 23.7
<code>ceil(x)</code>	arredonda x para o menor inteiro não menor que x	<code>ceil(9.2)</code> é 10.0 <code>ceil(-9.8)</code> é -9.0
<code>cos(x)</code>	cosseno trigonométrico de x (x em radianos)	<code>cos(0.0)</code> é 1.0
<code>exp(x)</code>	método exponencial e^x	<code>exp(1.0)</code> é 2.71828 <code>exp(2.0)</code> é 7.38906
<code>floor(x)</code>	arredonda x para o maior inteiro não maior que x	<code>floor(9.2)</code> é 9.0 <code>floor(-9.8)</code> é -10.0
<code>log(x)</code>	logaritmo natural de x (base e)	<code>log(Math.E)</code> é 1.0 <code>log(Math.E * Math.E)</code> é 2.0
<code>max(x,y)</code>	maior valor de x e y	<code>max(2.3, 12.7)</code> é 12.7 <code>max(-2.3, -12.7)</code> é -2.3
<code>min(x,y)</code>	menor valor de x e y	<code>min(2.3, 12.7)</code> é 2.3 <code>min(-2.3, -12.7)</code> é -12.7
<code>pow(x,y)</code>	x elevado à potência de y (isto é, x^y)	<code>pow(2.0, 7.0)</code> é 128.0 <code>pow(9.0, 0.5)</code> é 3.0
<code>sin(x)</code>	seno trigonométrico de x (x em radianos)	<code>sin(0.0)</code> é 0.0
<code>sqrt(x)</code>	raiz quadrada de x	<code>sqrt(900.0)</code> é 30.0
<code>tan(x)</code>	tangente trigonométrica de x (x em radianos)	<code>tan(0.0)</code> é 0.0

Classes Utilitárias Java



- Conjunto de classes e interfaces que implementam vários tipos de estruturas de dados
- Pacote `java.util` contém várias classes utilitárias

```
import java.util.Arrays;  
import java.util.Collections;  
import java.util.ArrayList;  
import java.util.Vector;  
import java.util.Stack;  
import java.util.HashMap;
```

Classes Utilitárias Java

Classe Arrays



- Esta classe fornece métodos `static` para manipulações de array comuns (vetores), incluindo:
 - `sort` para classificar um array (ordem crescente)
 - `binarySearch` para pesquisar em um array ordenado
 - `equal` para comparar arrays
 - `fill` para inserir valores em um array
- Esses métodos são sobrecarregados para arrays de tipos primitivos e arrays de objetos
- Sintaxe:
 - `tipo[] identVet; //apenas declaracao`
 - `tipo[] identVet = new tipo[qtde]; //decl+inic`

Exemplo Classe Arrays

```
1 // Figura 7.22: ArrayManipulations.java
2 // Métodos da classe Arrays e System.arraycopy.
3 import java.util.Arrays;
4
5 public class ArrayManipulations
6 {
7     public static void main(String[] args)
8     {
9         // classifica doubleArray em ordem crescente
10        double[] doubleArray = { 8.4, 9.3, 0.2, 7.9, 3.4 };
11        Arrays.sort(doubleArray);
12        System.out.printf("%ndoubleArray: ");
13
14        for (double value : doubleArray)
15            System.out.printf("%.1f ", value);
16
17        // preenche o array de 10 elementos com 7s
18        int[] filledIntArray = new int[10];
19        Arrays.fill(filledIntArray, 7);
20        displayArray(filledIntArray, "filledIntArray");
21
22        // copia array intArray em array intArrayCopy
23        int[] intArray = { 1, 2, 3, 4, 5, 6 };
24        int[] intArrayCopy = new int[intArray.length];
25        System.arraycopy(intArray, 0, intArrayCopy, 0, intArray.length);
26        displayArray(intArray, "intArray");
27        displayArray(intArrayCopy, "intArrayCopy");
28
29        // verifica a igualdade de intArray e intArrayCopy
30        boolean b = Arrays.equals(intArray, intArrayCopy);
31        System.out.printf("%n%nintArray %s intArrayCopy%n",
32            (b ? "==" : "!="));
33    }
```



UNIVALI

Exemplo Classe Arrays

```
34 // verifica a igualdade de intArray e filledIntArray
35 b = Arrays.equals(intArray, filledIntArray);
36 System.out.printf("intArray %s filledIntArray%n",
37     (b ? "=" : "!="));
38
39 // pesquisa o valor 5 em intArray
40 int location = Arrays.binarySearch(intArray, 5);
41
42 if (location >= 0)
43     System.out.printf(
44         "Found 5 at element %d in intArray%n", location);
45 else
46     System.out.println("5 not found in intArray");
47
48 // pesquisa o valor 8763 em intArray
49 location = Arrays.binarySearch(intArray, 8763);
50
51 if (location >= 0)
52     System.out.printf(
53         "Found 8763 at element %d in intArray%n", location);
54 else
55     System.out.println("8763 not found in intArray");
56 }
57
58 // gera saída de valores em cada array
59 public static void displayArray(int[] array, String description)
60 {
61     System.out.printf("%n%s: ", description);
62
63     for (int value : array)
64         System.out.printf("%d ", value);
65 }
66 } // fim da classe ArrayManipulations
```

```
doubleArray: 0.2 3.4 7.9 8.4 9.3
filledIntArray: 7 7 7 7 7 7 7 7 7 7
intArray: 1 2 3 4 5 6
intArrayCopy: 1 2 3 4 5 6
```

```
intArray == intArrayCopy
intArray != filledIntArray
Found 5 at element 4 in intArray
8763 not found in intArray
```


Classes Utilitárias Java

ArrayList



- Esta classe fornece uma solução conveniente para alterar dinamicamente seu tamanho em tempo de execução
- É uma lista genérica que pode ser declarada para armazenar objetos de qualquer **classe!!**

```
ArrayList<String> nomes = new ArrayList<>();
```

```
ArrayList<Pessoa> pessoas = new ArrayList<>();
```

- Somente tipos não primitivos podem ser usados – logo dará erro se fizer:

```
ArrayList<int> erro = new ArrayList<>();
```

Classes Utilitárias Java

ArrayList



- Mas o Java fornece o mecanismo *boxing*, que permite que valores primitivos sejam empacotados como objetos para uso com classes genéricas:

```
ArrayList<Integer> integers;
```

- Ao inserir um valor `int` em `ArrayList<Integer>` tal valor é *empacotado* como um objeto `Integer`.
- Logo, ao obter um objeto `Integer` de um `ArrayList<Integer>` e atribuí-lo a uma variável `int`, o valor `int` dentro do objeto é *desempacotado*.

Classes Utilitárias Java

ArrayList



- Alguns métodos e propriedades desta classe:
 - `boolean add(Object element)` adiciona o elemento especificado no final da lista.
 - `void add(int index, Object element)` insere o elemento especificado na posição indicada da lista.
 - `void clear()` : Remove todos os elementos da lista.
 - `boolean contains(Object element)` retorna verdadeiro se a lista contém o elemento especificado e falso caso contrário.
 - `Object get(int index)` retorna o i-ésimo elemento da lista.
 - `int indexOf(Object element)` retorna a posição da primeira ocorrência do elemento especificado na lista.

Classes Utilitárias Java

ArrayList



- Alguns métodos e propriedades desta classe:
 - `boolean isEmpty()` retorna verdadeiro se a lista estiver vazia e falso caso contrário.
 - `int lastIndexOf(Object element)` retorna a posição da última ocorrência do elemento especificado na lista.
 - `Object remove(int index)` remove o i-ésimo elemento da lista.
 - `Object set(int index, Object element)` substitui o i-ésimo elemento da lista pelo elemento especificado.
 - `int size()` retorna o número de elementos da lista.

Exemplo Classe ArrayList

```
1 // Figura 7.24: ArrayListCollection.java
2 // Demonstração da coleção ArrayList<T> genérica.
3 import java.util.ArrayList;
4
5 public class ArrayListCollection
6 {
7     public static void main(String[] args)
8     {
9         // cria um novo ArrayList de strings com uma capacidade inicial de 10
10        ArrayList<String> items = new ArrayList<String>(10);
11
12        items.add("red"); // anexa um item à lista
13        items.add(0, "yellow"); // insere "yellow" no índice 0
14
15        // cabeçalho
16        System.out.print(
17            "Display list contents with counter-controlled loop:");
18
19        // exibe as cores na lista
20        for (int i = 0; i < items.size(); i++)
21            System.out.printf(" %s", items.get(i));
22
23        // exibe as cores usando for aprimorada no método display
24        display(items,
25            "%nDisplay list contents with enhanced for statement:");
26
27        items.add("green"); // adiciona "green" ao fim da lista
28        items.add("yellow"); // adiciona "yellow" ao fim da lista
29        display(items, "List with two new elements:");
30
31        items.remove("yellow"); // remove o primeiro "yellow"
32        display(items, "Remove first instance of yellow:");
33    }
```

Exemplo Classe ArrayList

```
34 items.remove(1); // remove o item no índice 1
35 display(items, "Remove second list element (green):");
36
37 // verifica se um valor está na List
38 System.out.printf("\n\"red\" is %sin the list\n",
39 items.contains("red") ? "" : "not ");
40
41 // exibe o número de elementos na List
42 System.out.printf("Size: %s\n", items.size());
43 }
44
45 // exibe elementos do ArrayList no console
46 public static void display(ArrayList<String> items, String header)
47 {
48     System.out.printf(header); // exibe o cabeçalho
49
50     // exibe cada elemento em itens
51     for (String item : items)
52         System.out.printf(" %s", item);
53
54     System.out.println();
55 }
56 } // fim da classe ArrayListCollection
```

```
Display list contents with counter-controlled loop: yellow red
Display list contents with enhanced for statement: yellow red
List with two new elements: yellow red green yellow
Remove first instance of yellow: red green yellow
Remove second list element (green): red yellow
"red" is in the list
Size: 2
```